



Знакомство с Git



План

- Version Control и его история
- Принципы работы с Git
- Основы работы с Git. Основные команды

Version Control

- Единый "источник правды"
- Основа для коллективной работы
- Основа процесса разработки ПО и всего процесса непрерывной поставки в целом
- Основа для DevOps-практик



git



GITHUB

DROPBOX



v1.0.0



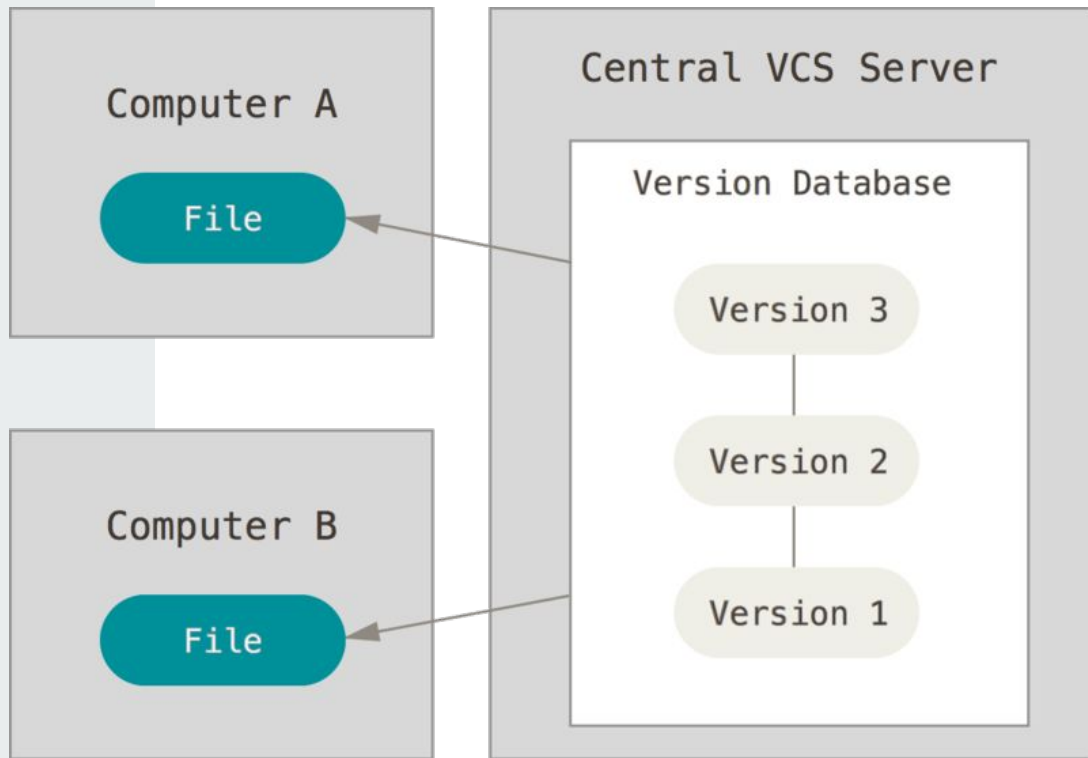
v1.0.1



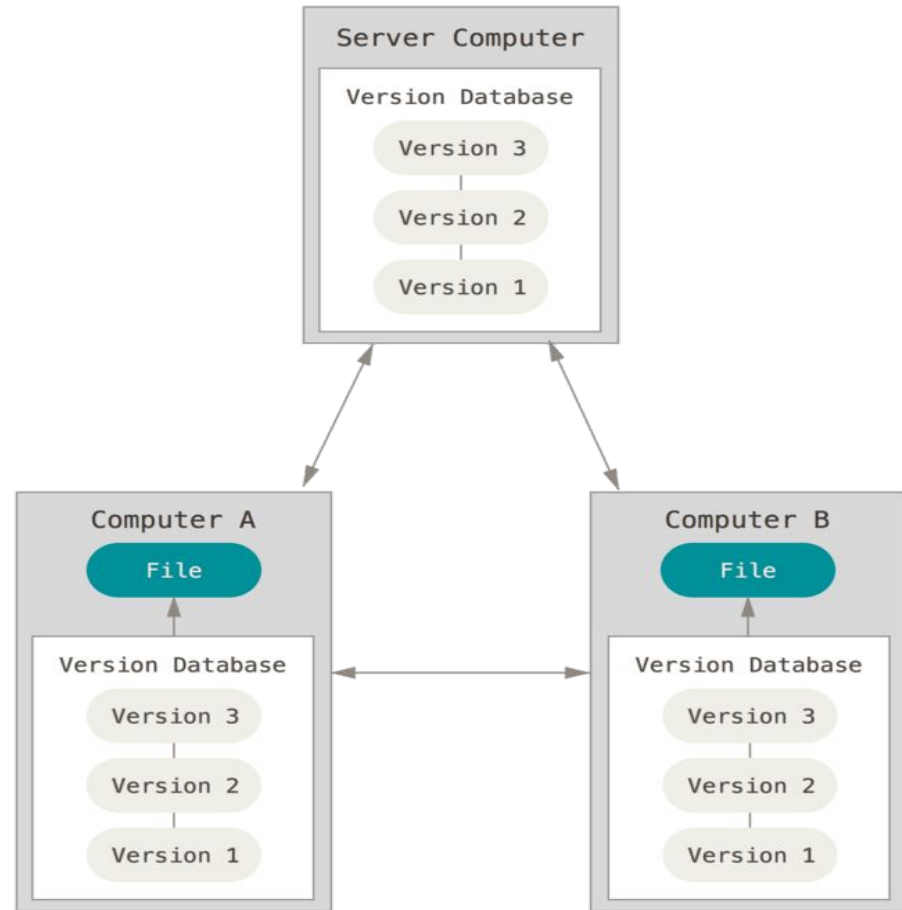
Зачем нужен Version Control?

- Возврат к старым версиям (safety net)
- Отслеживание изменений
- Хранение истории проекта
- Совместная работа
- Кто, когда и зачем?

Централизованные VCS

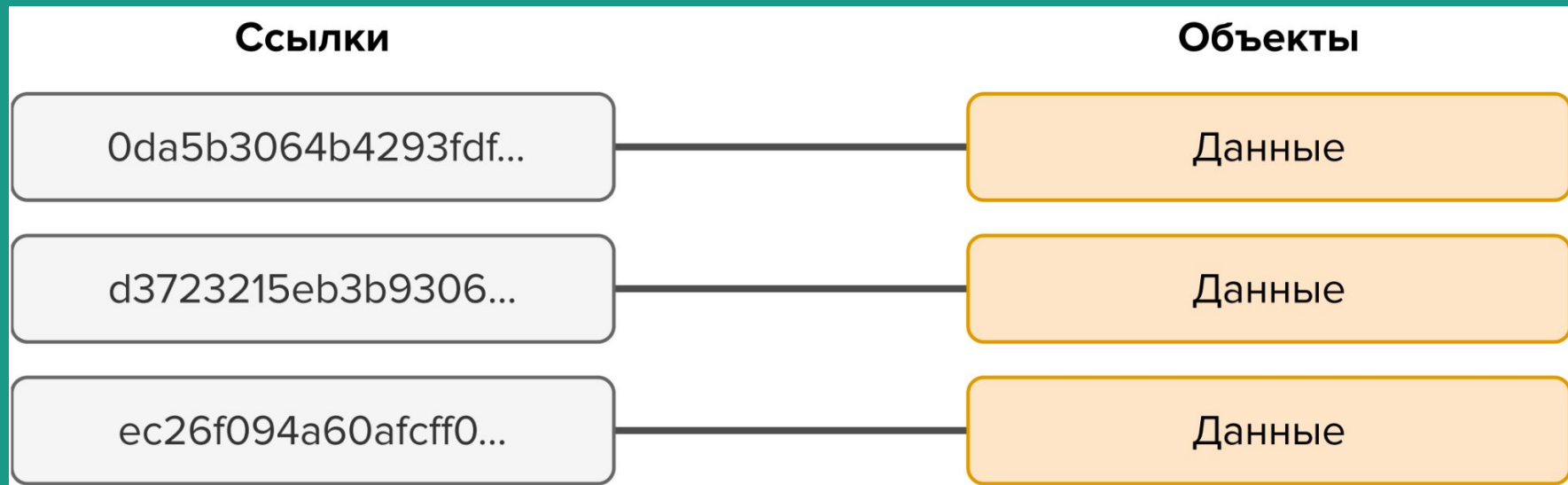


Распределенные VCS

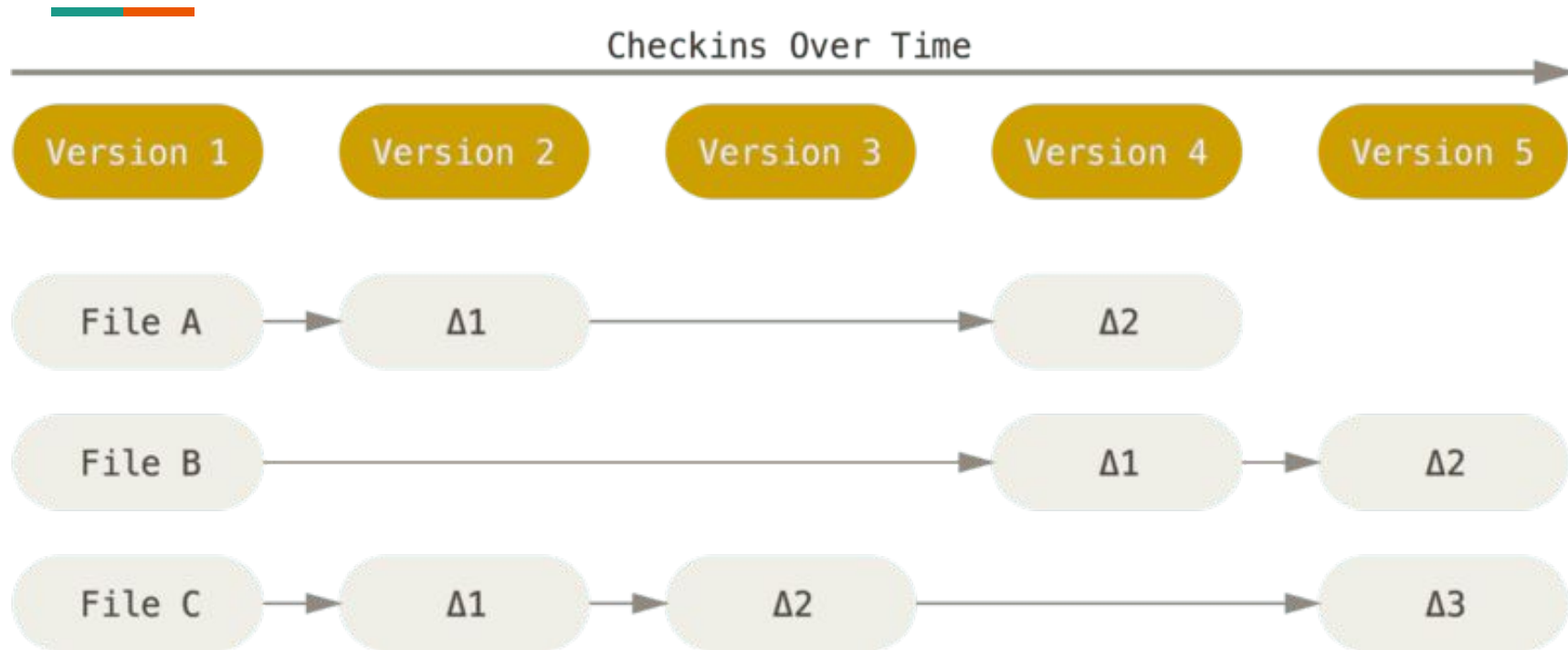




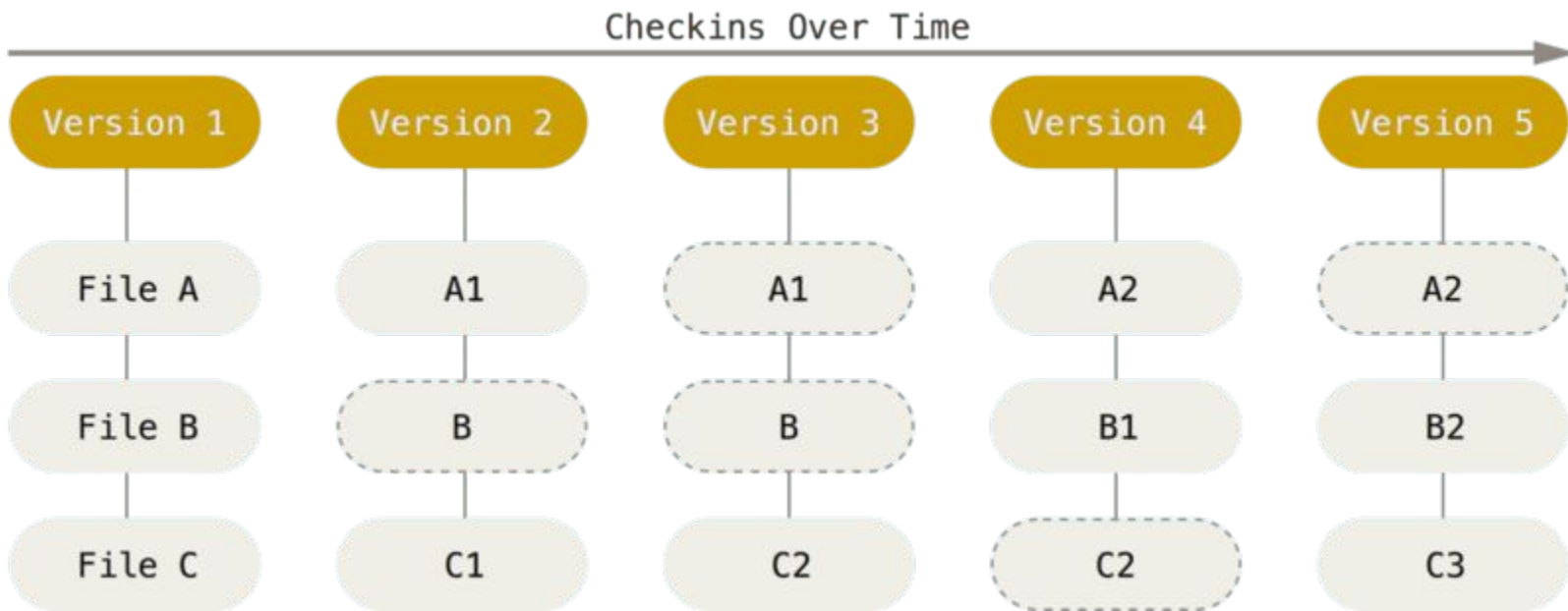
Хранилище с адресацией по содержимому



Хранение изменений (Subversion)



Хранение изменений (Git)



Где хранится история?

Git-репозиторий хранится в директории

.git в корне проекта.

В репозитории содержатся:

- объекты коммитов
- ссылки на коммиты и ветки
- конфигурация
- скрипты-хуки

```
.git
├── COMMIT_EDITMSG
├── FETCH_HEAD
├── HEAD
├── config
├── description
├── hooks
├── index
├── info
├── logs
│   ├── HEAD
│   └── refs
│       └── heads
├── objects
│   ├── 01
│   │   └── 2dc51bccc1
│   ├── 5a
│   │   └── 2669f47b80
│   ├── 7b
│   │   ├── bcb3154035
│   │   └── cf9b7d210c
│   ├── 9d
│   │   └── 4fc2f6782e
│   ├── f7
│   │   └── a9d2e60188
│   ├── fd
│   │   └── 96eb24c77b
│   ├── info
│   └── pack
└── refs
    ├── heads
    │   └── master
    └── tags
```

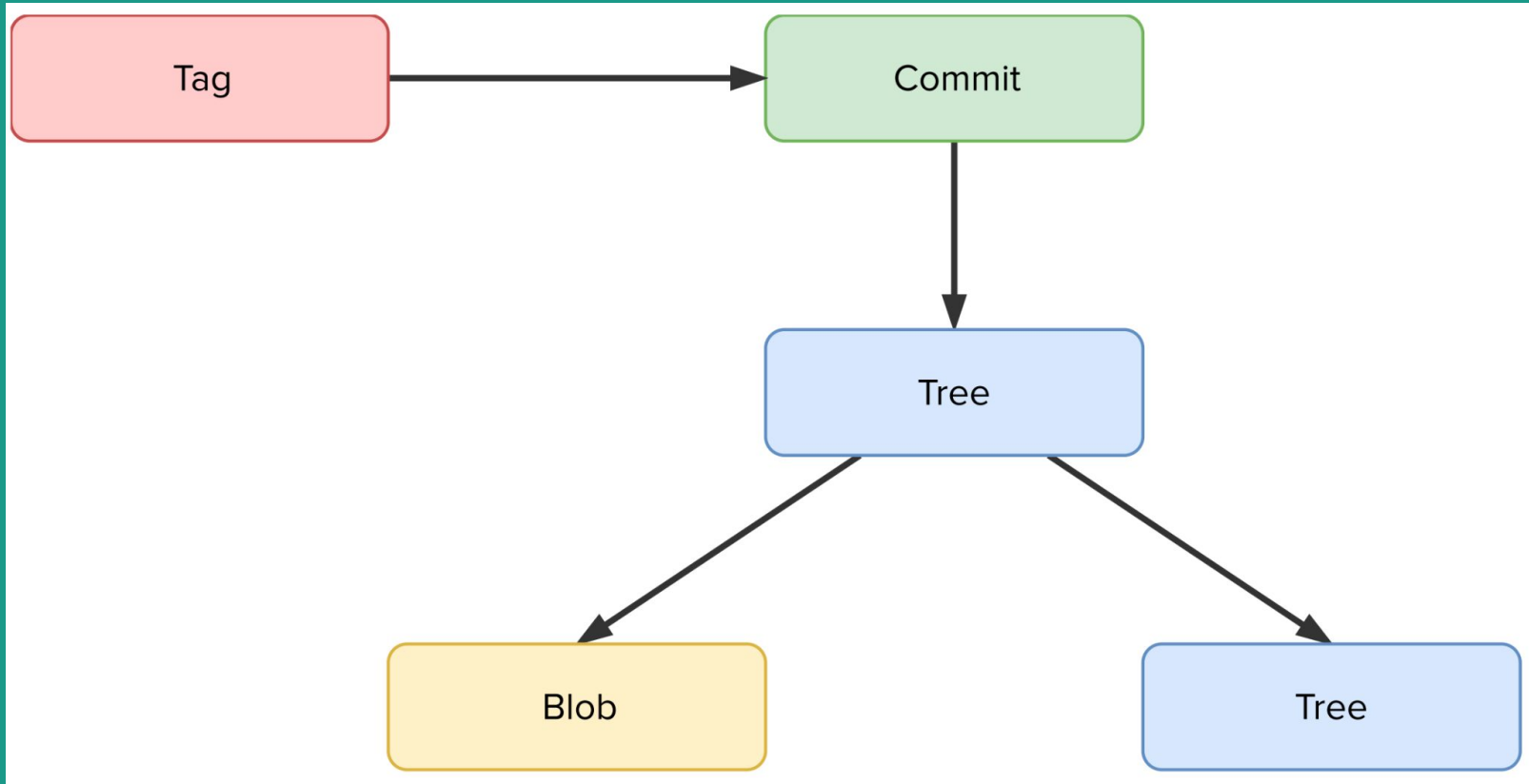


Как хранятся изменения?

В репозитории есть 4
типа хранимых
объектов:

1. Blob
2. Tree
3. Commit
4. Tag

Структура хранимых данных



Данные (blob)

Blob - базовая единица хранения данных в Git.

Хранит snapshot (снимок) содержимого файла.

В качестве имени объекта берется SHA1 хеш, содержимого файла и заголовка.

```
~/src/repo [master]$ git cat-file
-t 8c7a1cf
blob
~/src/repo [master]$ git cat-file
-p 8c7a1cf
this is file.txt
second commit did this
~/src/repo [master]$ cat file.txt
this is file.txt
second commit did this
```

tree (дерево)

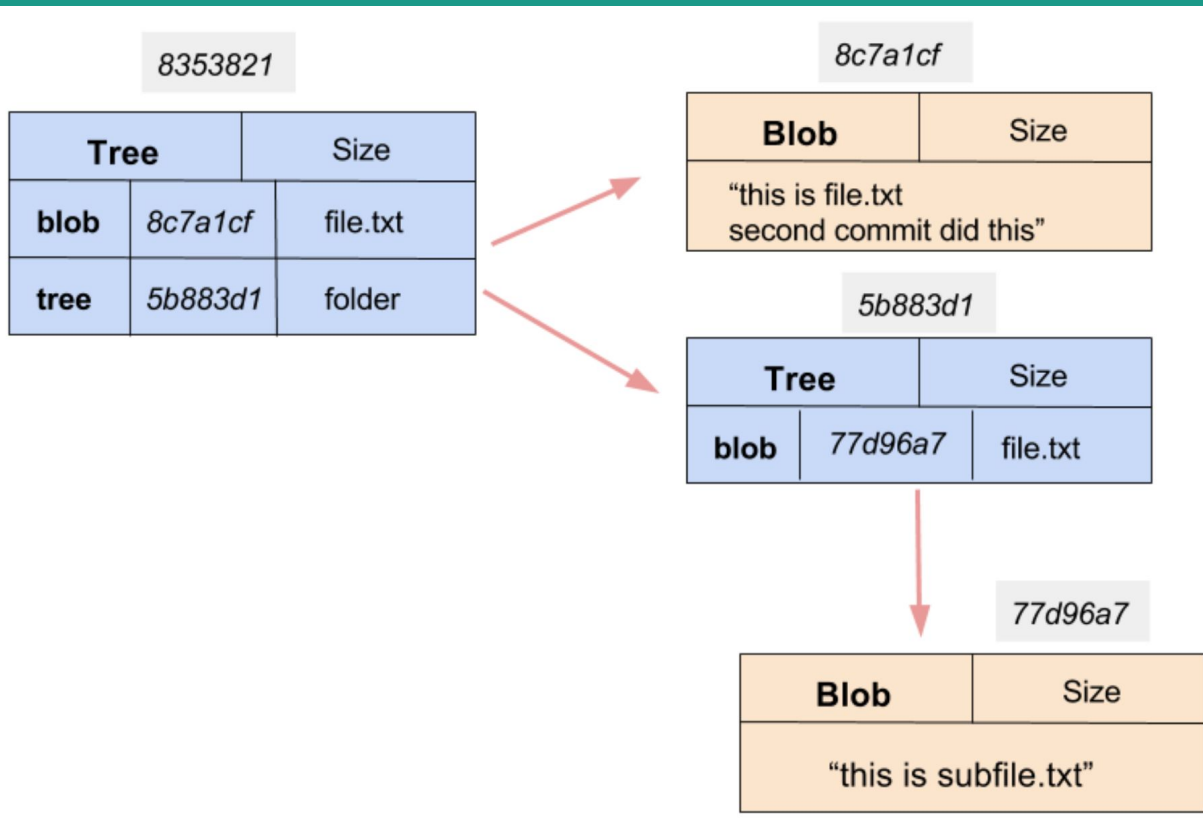


Деревья содержат информацию о блобах, а также других поддеревьях. Решают проблему хранения имен файлов и их группировки по директориям.

```
.  
├── file.txt  
├── folder  
└── subfile.txt
```

```
~/src/repo [master]$ git cat-file -t  
8353821  
tree  
~/src/repo [master]$ git ls-tree 8353821  
100644 blob  
8c7a1cf844966a95bf68faf3cdd9da7f3d7662be  
file.txt  
040000 tree  
5b883d12e60cfc52016d9dc9515e28b9e43850b7  
folder  
~/src/repo [master]$ git ls-tree 5b883d1  
100644 blob  
77d96a7ca3be9b02eb1d3624d1526d2a0a1ce4d5  
subfile.txt
```

Blobs и Trees

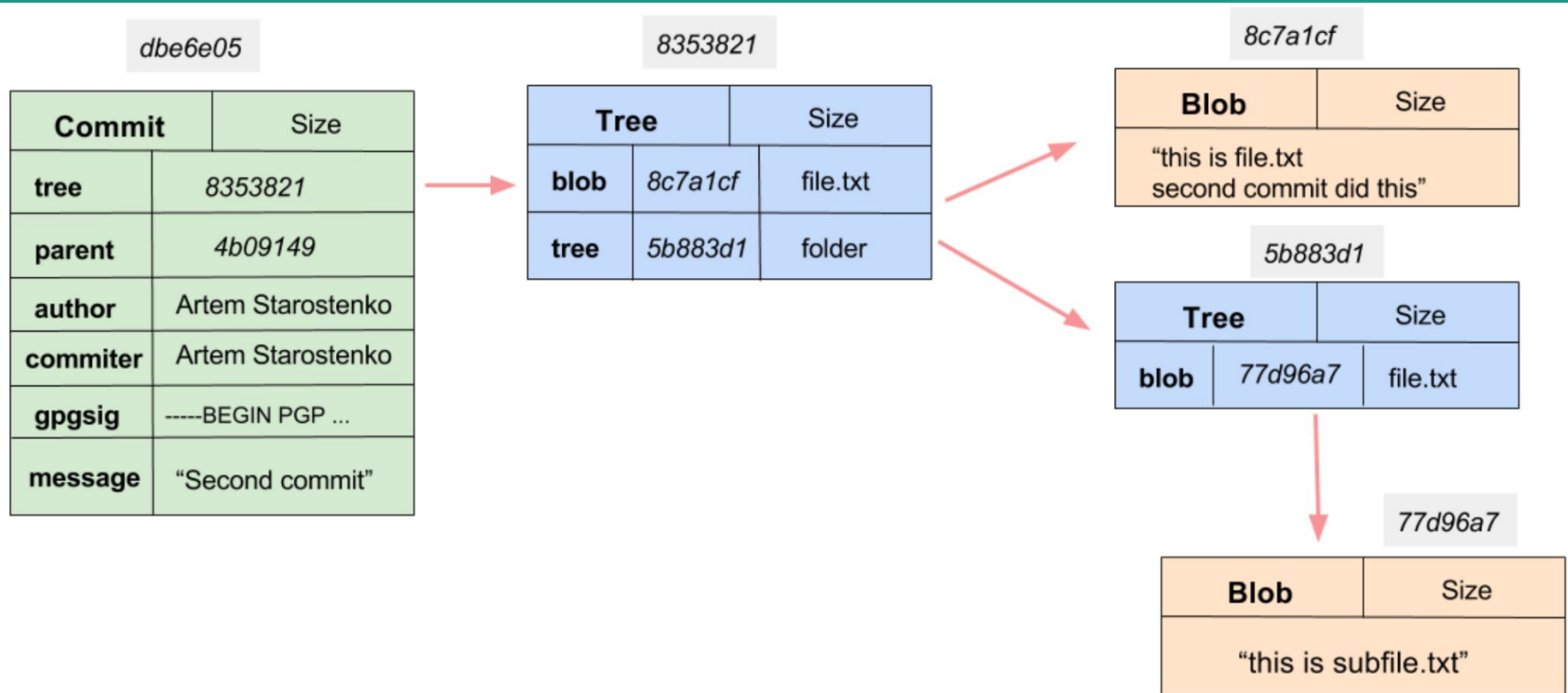


Commit



- Автор, Коммитер, Дата
- Цифровая подпись
- Сообщение
- Дерево
- Родитель (или родители)

Commit



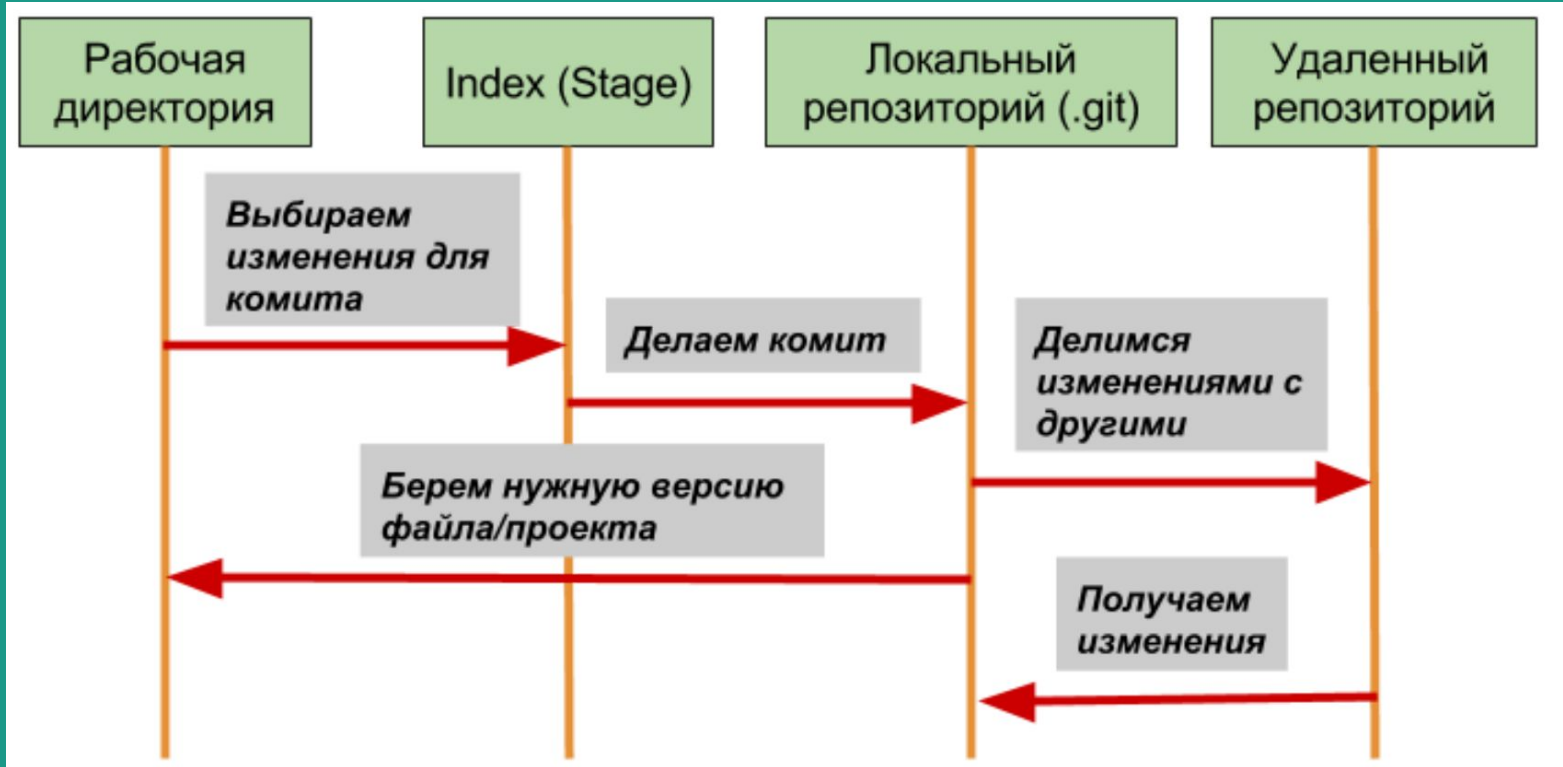
Tags aka Тэги



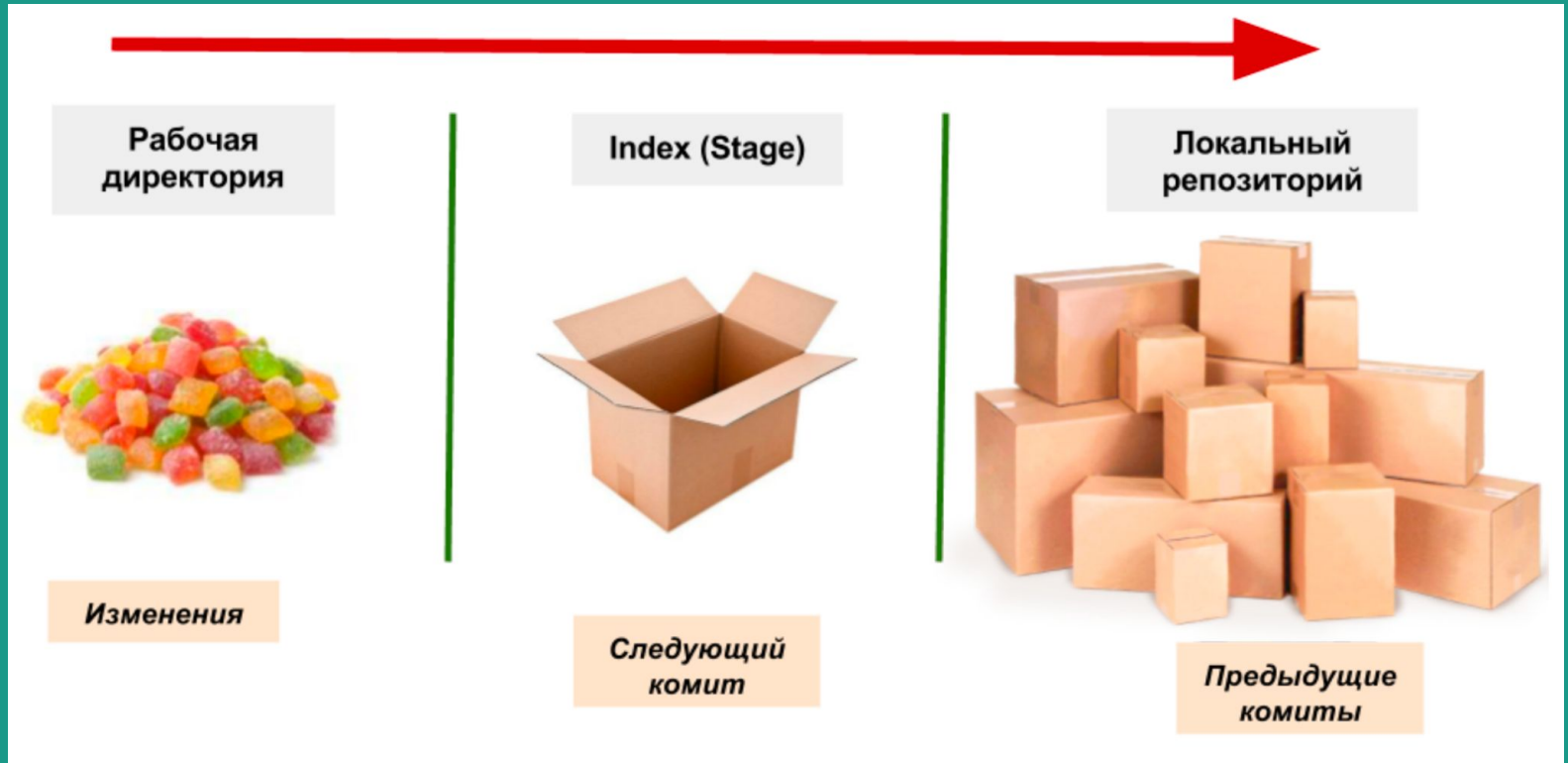
Два типа:

- Легковесные теги - указатели на определенный коммит
- Аннотированные - содержат имя, email поставившего тег, дату, комментарий и могут иметь цифровую подпись. Хранятся как полноценные объекты.

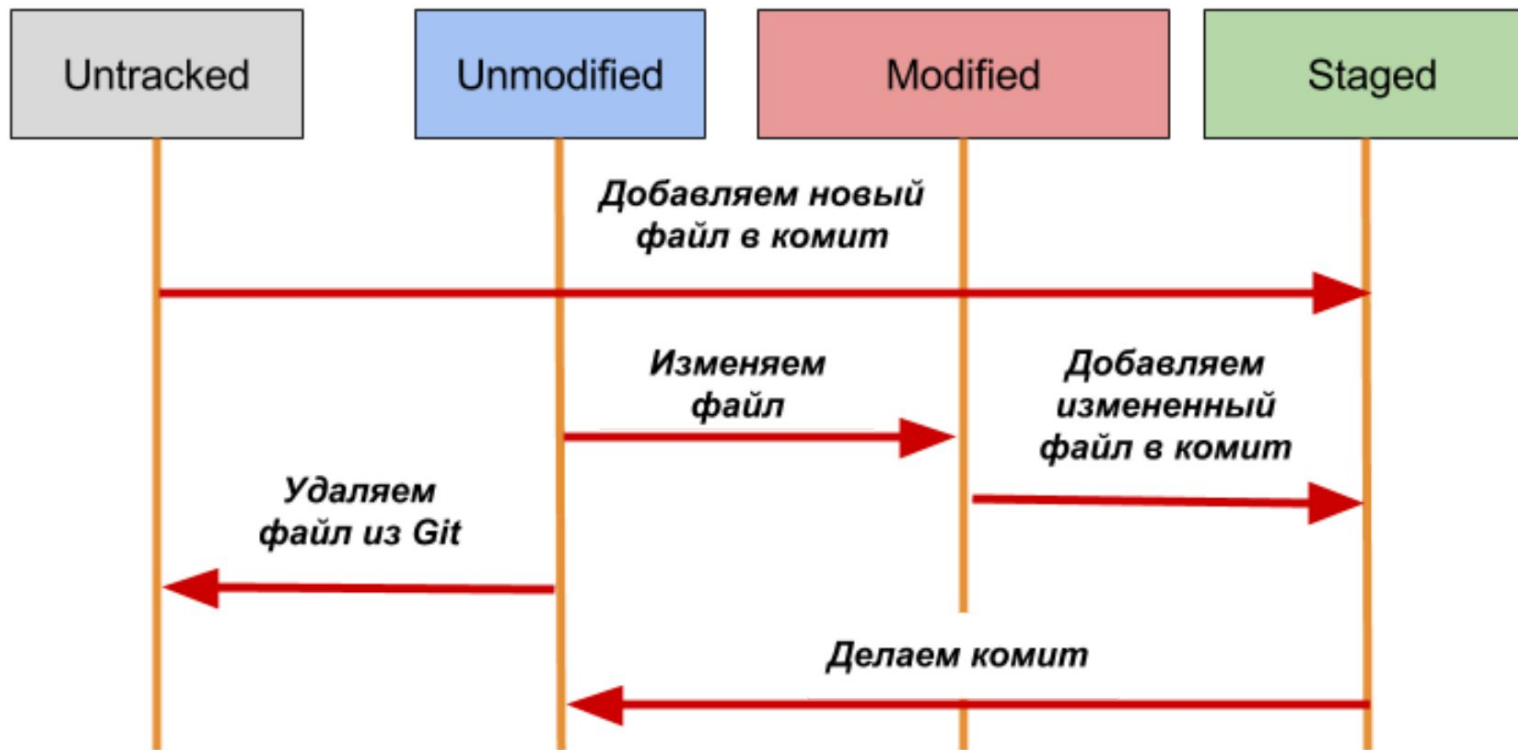
Работа с Git



Работа с Git



Состояния файлов





Основные команды для работы с Git

Конфигурация Git

Системные

(настройки на уровне всей системы)

Команда:

```
git config --system
```

Глобальные

(настройки конкретного пользователя)

Команда:

```
git config --global
```

Локальные

(настройки конкретного репозитория)

Команда:

```
git config --local
```

Чтобы посмотреть все установленные настройки и узнать где именно они заданы, используйте команду:

```
$ git config --list --show-origin
```



Основные секции

Самое важное, это настроить здесь имя и e-mail, так как данная информация будет использоваться при создании коммитов.

```
ilfatgaliulin@Ilfats-MacBook-Air ~$ cat .gitconfig
[user]
    name = ilfat.galiulin
    email = galiulinilfat@gmail.com
[filter "lfs"]
    clean = git-lfs clean -- %f
    smudge = git-lfs smudge -- %f
    process = git-lfs filter-process
    required = true
[alias]
    st = status
```


Создание Git репозитория



Инициализация нового Git-репозитория локально с помощью команды `git init`

Клонирование уже существующего репозитория с помощью команды `git clone`

Git status

Отображает список всех изменений,
которые имеются на данный
момент и не закоммичены

```
ilfatgaliulin@Ilfats-MacBook-Air ~/docker_tutorial/mintrans/docker } dev-oldtransport touch docker-compose.override.yml
ilfatgaliulin@Ilfats-MacBook-Air ~/docker_tutorial/mintrans/docker } dev-oldtransport git status
Текущая ветка: dev-oldtransport
Эта ветка соответствует «origin/dev-oldtransport».
```

Неотслеживаемые файлы:

(используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)

`docker-compose.override.yml`

индекс пуст, но есть неотслеживаемые файлы

(используйте «git add», чтобы проиндексировать их)

Файл .gitignore

Позволяет явно объявить
какие имена файлов не
нужно
отслеживать. Можно
использовать маски

```
deepak@ubuntu:~/git_examples$ git status --ignored
On branch new-feature
Your branch is up to date with 'origin/new-feature'.

Ignored files:
(use "git add -f <file>..." to include in what will be committed)
templates/test/
test/critical_file
test/file1.log1
test/file1.log12
test/file1.login
test/file2
test/important_file
nothing to commit, working tree clean
```

ну да, ну да

Как сделать коммит ?

Добавить только нужные нам файлы (или все подряд через **git add .**) и потом сделать коммит:

```
git add <filename>
```

```
git commit -m "Ой, я, кажется, накомитил"
```

Добавить в staging все отслеживаемые изменения файлов и сразу сделать коммит:

```
git commit -am "My short commit message"
```

Best Practices при работе с сообщениями

Сообщение в каждом коммите должно соответствовать следующим правилам:

- Должно содержать описание целей или причин внесения изменений
- Должно содержать номер задачи в трекере (Jira, GitHub Issues)
- Должно быть содержательным и кратким. "Bugfix", "Ooops", "Commit" - не пояснение к коммиту

Как исправить КОММИТ



Команда

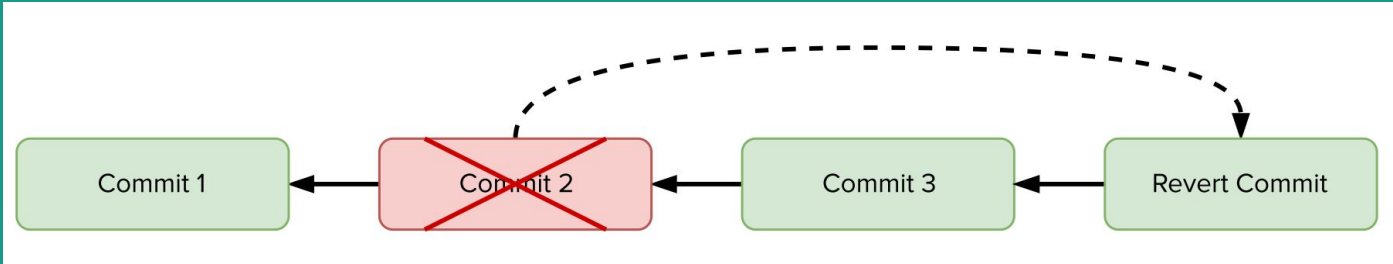
`git commit --amend`

Позволяет исправить
последний коммит (изменить
сообщение или добавить
файлы):

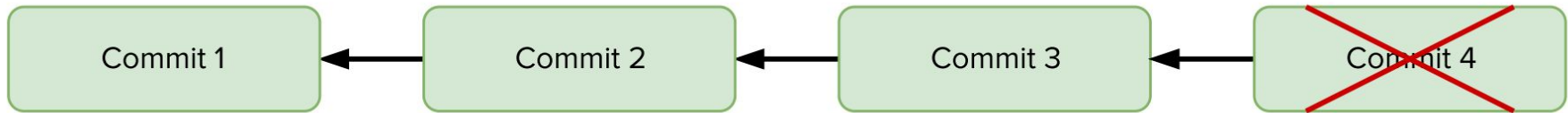
```
~/src/repo [master*]$ git commit -m "bad commit  
message"  
[master 3122367] bad commit message  
1 file changed, 0 insertions(+), 0 deletions(-)  
create mode 100644 n.txt  
~/src/repo [master]$ git commit --amend -m  
"good commit message"  
[master 6ec9633] good commit message  
Date: Wed Apr 26 02:29:13 2017 +0300  
1 file changed, 0 insertions(+), 0 deletions(-)  
create mode 100644 n.txt
```

Как отменить коммит

Команда **git revert** делает новый (обратный) коммит. Не меняет истории - безопасный способ откатить изменения. Стоит использовать для публичных коммитов.



Команда **git reset HEAD~1** удаляет коммиты после указанного. Меняет историю - небезопасный способ откатить изменения. Можно использовать для локальных коммитов.



Поиск по сообщению

Команда `git log --grep=XXX` позволяет выполнять поиск по сообщениям в коммитах:

```
commit 4e40b4603e6558ef7547888ac2aed47f0623c6e4 (HEAD -> dev-oldtransport, origin/dev-oldtransport)
Author: Maksim Redkin <maksim.redkin@tatar.ru>
Date:   Wed Feb 22 16:53:34 2023 +0300

    many fixes

commit 97b95be81fc78894b090c16ad425bb7a45f6aa4c
Author: Maksim Redkin <maksim.redkin@tatar.ru>
Date:   Wed Feb 22 16:38:25 2023 +0300

    add traefik

commit 94c56569ce3d818ff82a39f6830723424bc96b55
Author: Maksim Redkin <maksim.redkin@tatar.ru>
Date:   Wed Feb 22 16:35:59 2023 +0300

    update build-push.sh

commit c420f6148d51b3930fa2869034daced225a6d222
Author: Ilfat Galiulin <Ilfat.Galiulin@tatar.ru>
Date:   Wed Feb 22 16:18:22 2023 +0300

    changes in dockerfiles

commit 4b3f832d26b6047aafe51e3b358569e10bf4147e
Author: Maksim Redkin <maksim.redkin@tatar.ru>
Date:   Tue Feb 21 16:47:26 2023 +0300

    update docker-compose file

commit b26ccca49f9e249c42abe85e72185b8ca1a359a6
Author: Ilfat Galiulin <Ilfat.Galiulin@tatar.ru>
Date:   Fri Jan 27 17:29:01 2023 +0300

    dev branch for app in docker

commit 0d810830c8d3362392a2a0c40fc77828f97fe0ad (origin/oldtransport)
Author: Albert Garifov <albert.garifov@tatar.ru>
Date:   Wed Nov 30 13:44:58 2022 +0300

    fix
```


Ветки



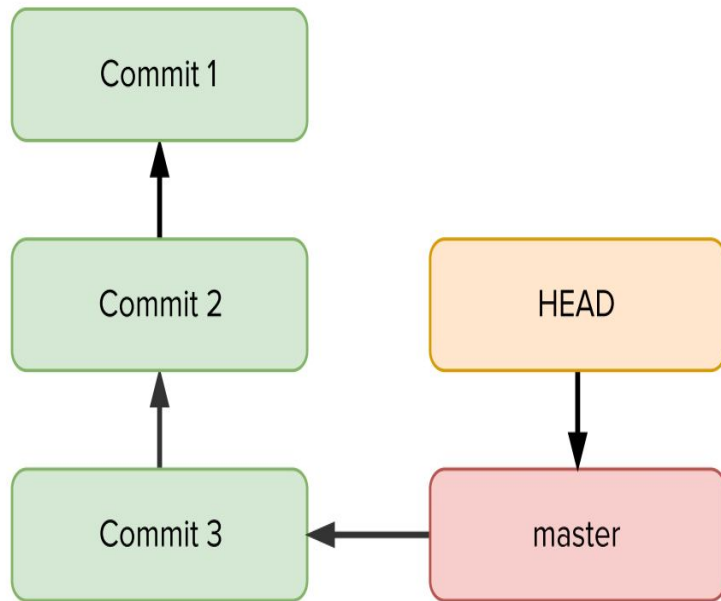


Ветки

- Подвижный указатель на коммит
- Переключение по веткам осуществляется за счет другого подвижного указателя: HEAD (указывает на текущий коммит)
- Текущий указатель автоматически сдвигается после каждого нового коммита

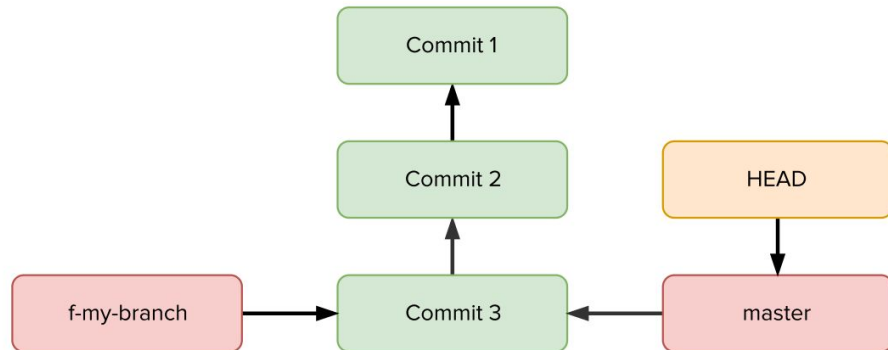
Ветка master

По умолчанию создается
ветка **master** (main):



Ветка feature

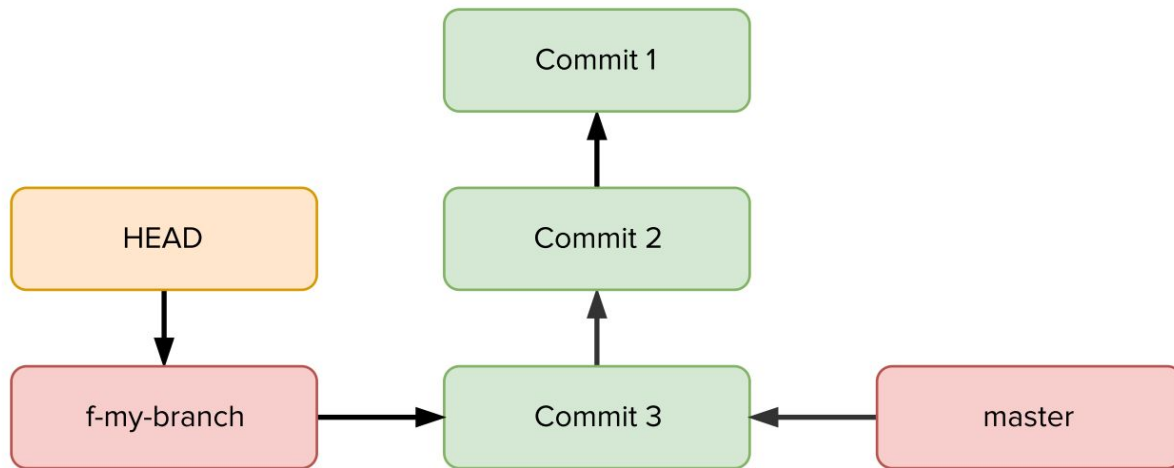
Результат создания ветки
командой `git branch`
`f-my-branch`:



Git Checkout

Указатель HEAD можно
переместить командой:

```
git checkout f-my-branch:
```



Работа с ветками

- Создать ветку:

```
git branch <branch_name>
```

- Переключиться на ветку

```
git checkout <branch_name>
```

- Создать ветку и сразу перейти в нее:

```
git checkout -b <branch_name>
```

Текущая ветка



Текущая положение
указателя HEAD
можно посмотреть в
файле
.git/HEAD:

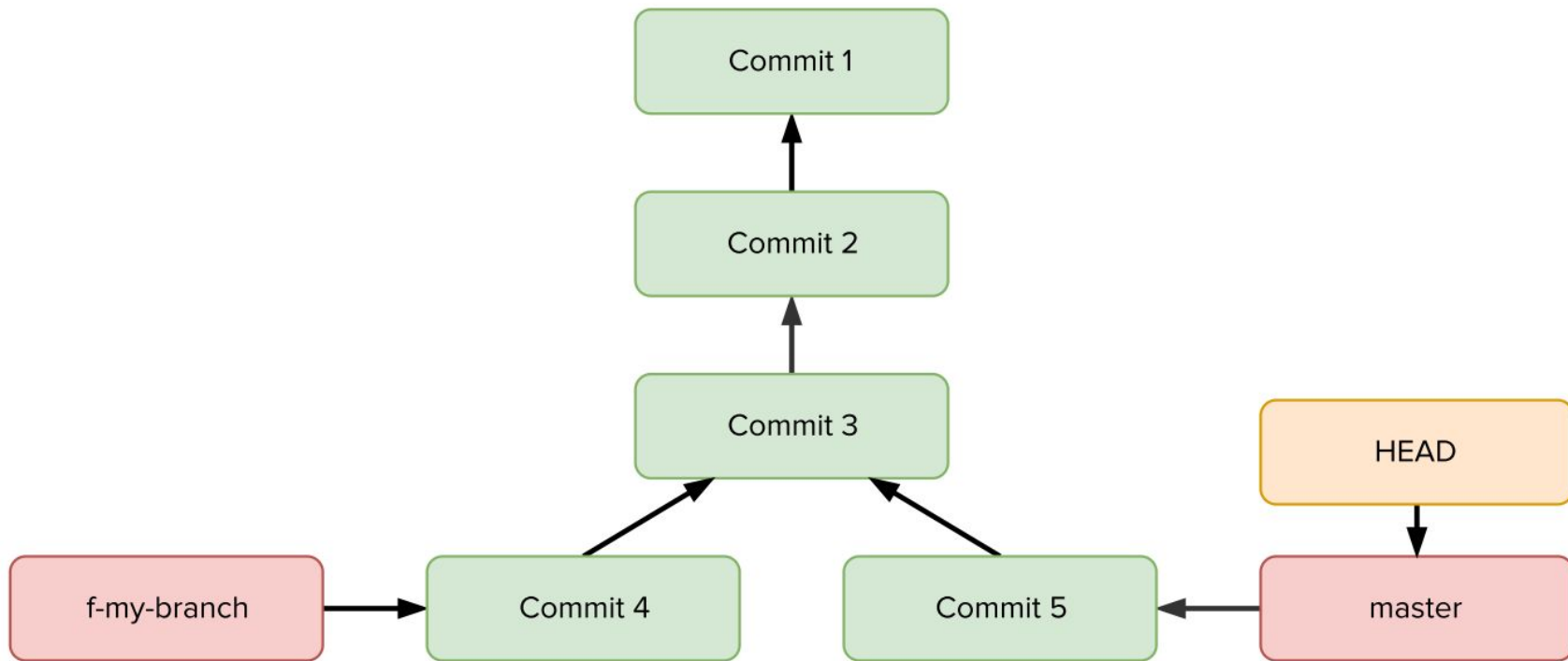
```
~/src/repo [new-feature]$ cat  
.git/HEAD  
ref: refs/heads/new-feature
```

Ссылки на локальные ветки

В каталоге `.git/refs/heads`
перечислены локальные
ветки и
ССЫЛКИ НА КОММИТЫ В НИХ:

```
x ilfatgaliulin@Ilfats-MacBook-Air ~/docker_tutorial/mintrans dev-oldtransport ± ls -la .git/refs/heads
total 16
drwxr-xr-x  4 ilfatgaliulin  staff  128 15 map 23:34 .
drwxr-xr-x  5 ilfatgaliulin  staff  160 15 map 23:32 ..
-rw-r--r--  1 ilfatgaliulin  staff   41 15 map 23:34 dev-oldtransport
-rw-r--r--  1 ilfatgaliulin  staff   41 15 map 23:32 master
ilfatgaliulin@Ilfats-MacBook-Air ~/docker_tutorial/mintrans dev-oldtransport ± ls -la .git/refs/heads/dev-oldtransport
-rw-r--r--  1 ilfatgaliulin  staff   41 15 map 23:34 .git/refs/heads/dev-oldtransport
ilfatgaliulin@Ilfats-MacBook-Air ~/docker_tutorial/mintrans dev-oldtransport ± cat .git/refs/heads/dev-oldtransport
4e40b4603e6558ef7547888ac2aed47f0623c6e4
ilfatgaliulin@Ilfats-MacBook-Air ~/docker_tutorial/mintrans dev-oldtransport ± cat .git/refs/heads/master
85722976088fb408f7ef95ab4422f50d1c643875
ilfatgaliulin@Ilfats-MacBook-Air ~/docker_tutorial/mintrans dev-oldtransport ±
```


Добавим коммиты в обе ветки

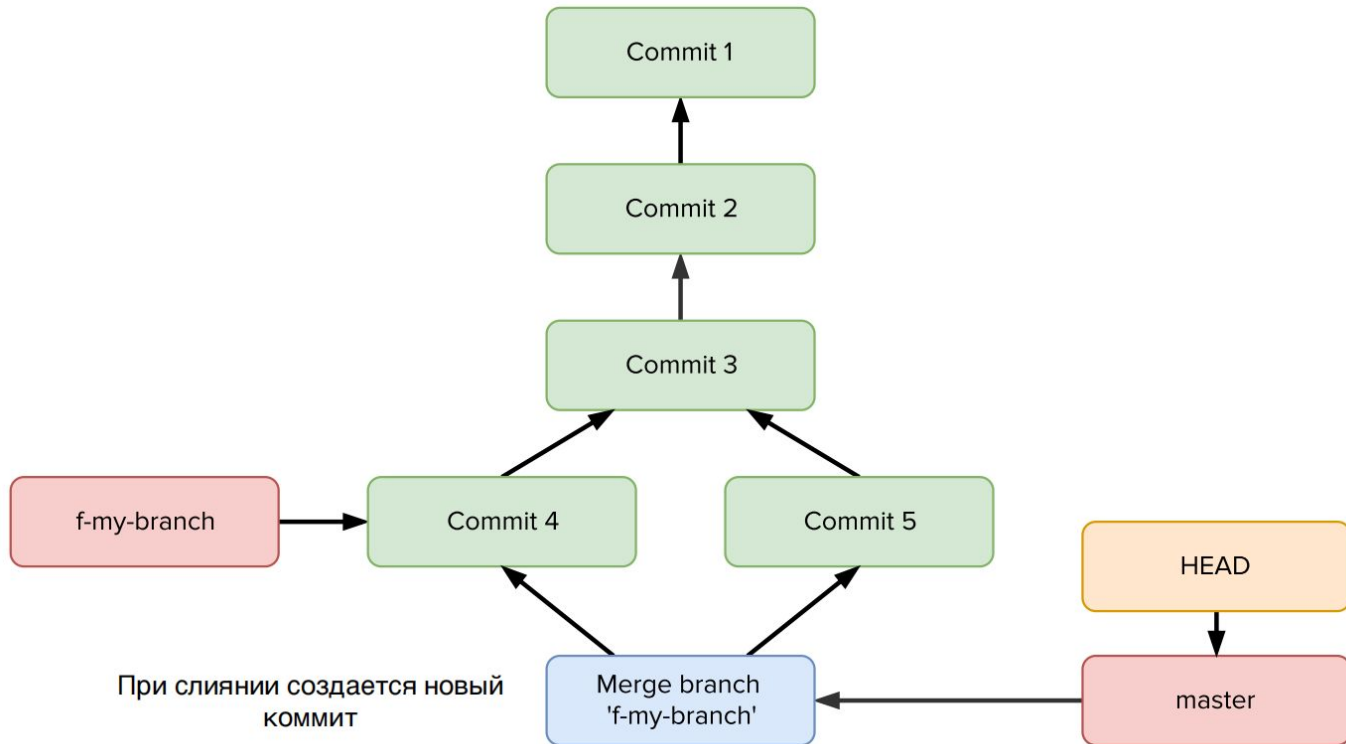


То же самое, но в ASCII:

```
~/src/repo [new-feature]$ git log --graph --abbrev-commit --decorate --all --oneline
* ea16714 (HEAD -> master) Commit5
| * f231481 (feature) Commit4
|/
* 0f72b9b Commit3
* ed0a3de Commit2
* 64a9ac9 Commit1
```

Git Merge | Перенос изменений

Добавляем изменения из одной ветки в другую:



Git Merge | CLI

Переходим на ветку **master**:

```
$ git checkout master
```

Сливаем ветку **f-my-branch** с веткой **master**:

```
$ git merge f-my-branch
```

```
* 7783eea (HEAD -> master) Merge branch 'f-my-branch'
```

```
|\
```

```
| * f231481 (f-my-branch) Commit4
```

```
* | ea16714 Commit5
```

```
|/
```

```
* 0f72b9b Commit3
```

```
* ed0a3de Commit2
```

```
* 64a9ac9 Commit1
```

Git Merge | Конфликты

- В разных ветках поменяли один и тот же файл в одном и том же месте
- При слиянии неясно, какие изменения применять, и Git не может автоматически объединить изменения
- Git помечает в файлах "спорные места" и эти конфликты необходимо устранить вручную

Git Merge | Конфликты

```
$ cat file.txt
КОММИТ 1
КОММИТ 2
<<<<<< HEAD
КОММИТ 4 (Я - КОММИТ ИЗ ВЕТКИ 'master')
=====
КОММИТ 3 (А я - КОММИТ ИЗ ВЕТКИ 'f-my-branch')
>>>>>> f-my-branch
$ vim file.txt
КОММИТ 1
КОММИТ 2
КОММИТ 3
КОММИТ 4
$ git add file.txt
$ git commit -m "Merge branch feature"
```

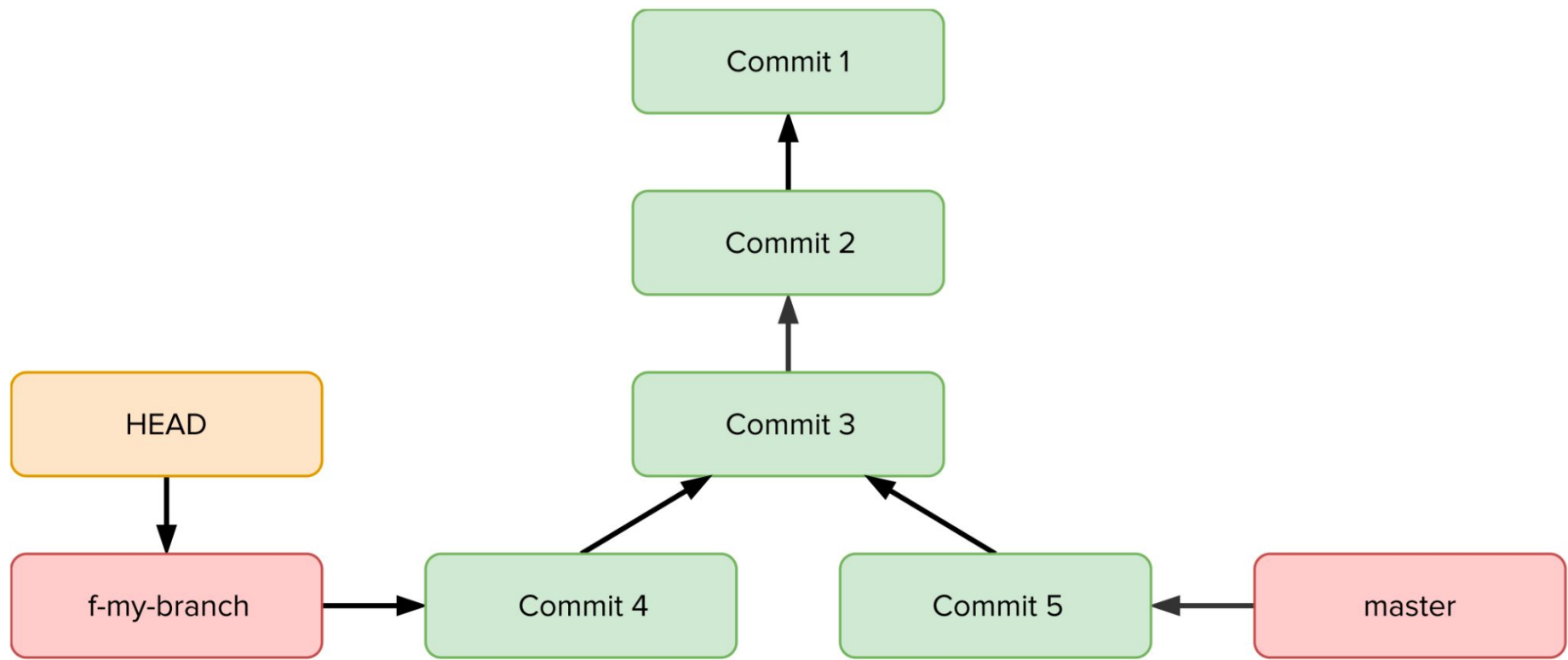
git rebase



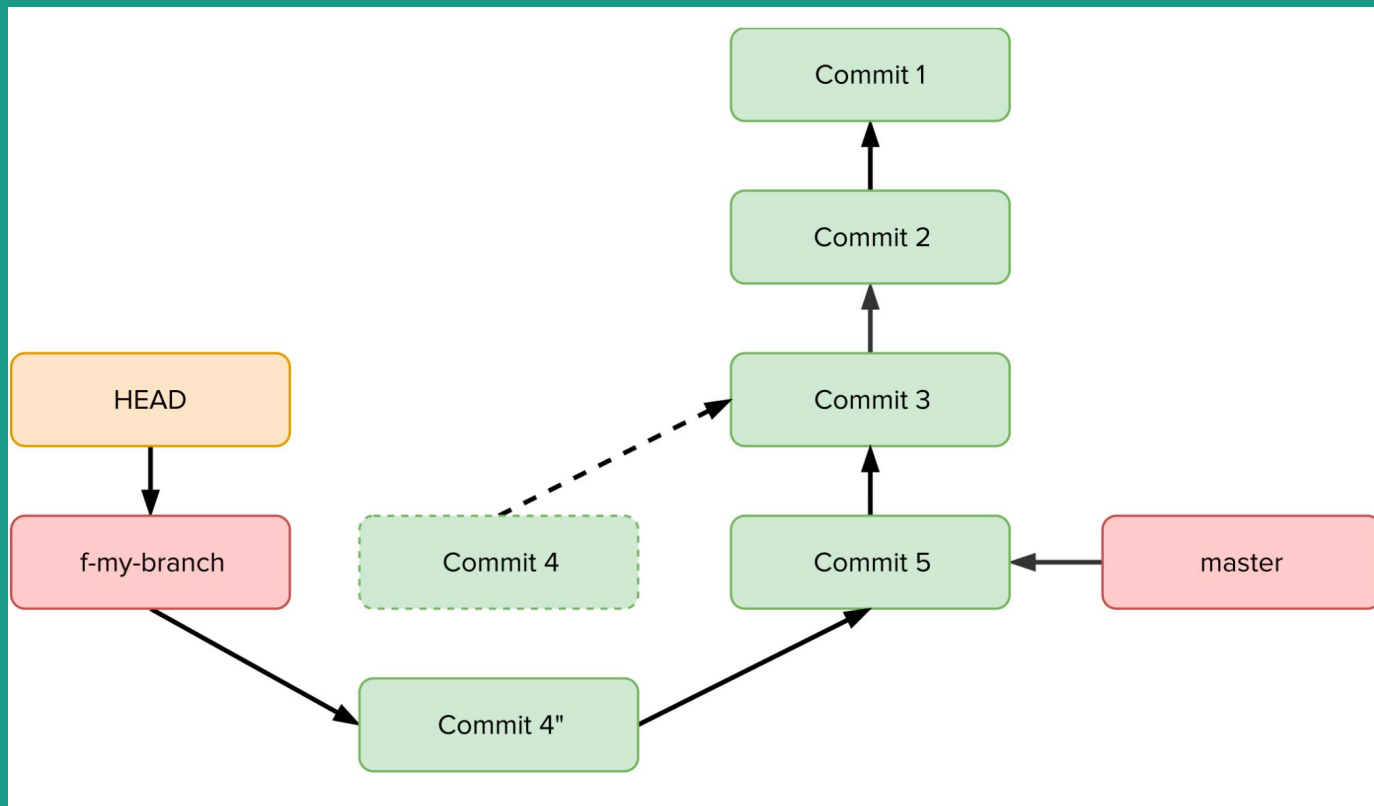
Команда `git rebase` меняет основание текущей ветки и позволяет "исправлять историю".

- Переходим на ветку f-my-branch: `git checkout f-my-branch`
- Меняем основание ветки f-my-branch: `git rebase master`

До rebase



После git rebase master



Git Rebase CLI



- До **rebase**:

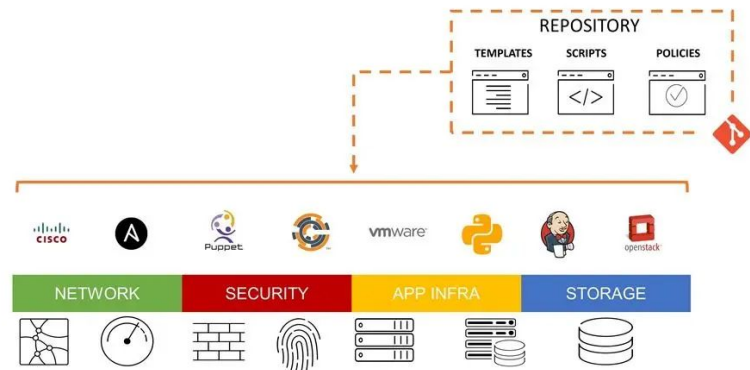
```
* ea16714 (master) Commit5
| * f231481 (HEAD -> f-my-branch)
Commit4
|/
* 0f72b9b Commit3
* ed0a3de Commit2
* 64a9ac9 Commit1
```

- После **rebase**:

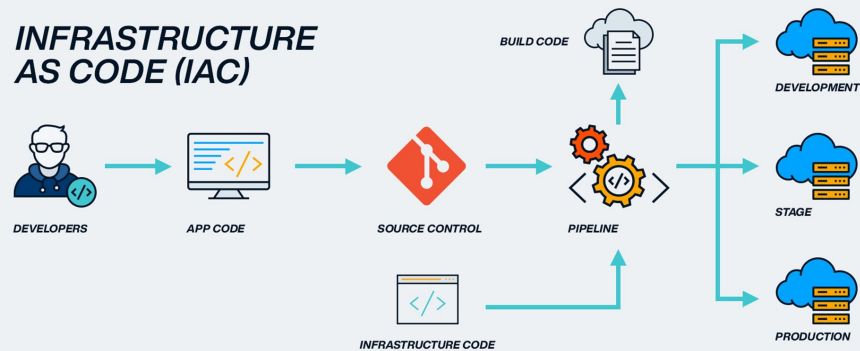
```
* 34bf60f (HEAD -> f-my-branch)
Commit4
* ea16714 (master) Commit5
* 0f72b9b Commit3
* ed0a3de Commit2
* 64a9ac9 Commit1
```

Git = IaC

INFRASTRUCTURE as CODE



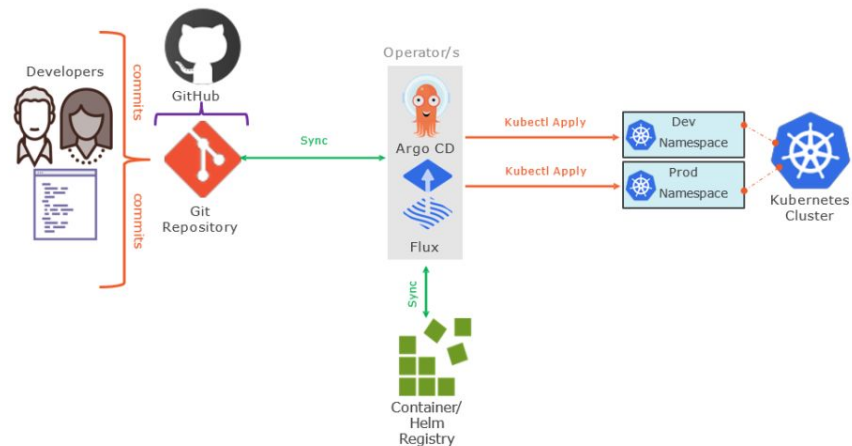
INFRASTRUCTURE AS CODE (IAC)



В Git живёт GitOps



GitOps Architecture





**Для SRE Git — это
инструмент
постмортемов**

